

JaVers Auditing with Spring Boot, JPA & MySQL

Professional 2–3 Page Implementation Guide | Example: Request / Time-Off Module

1. Introduction & Purpose

JaVers is a Java object-auditing and versioning framework that records how domain objects change over time. With Spring Data JPA integration, JaVers can automatically create audit snapshots when audited repository operations occur. This is useful when an application needs to answer: what changed, what was the previous value, what is the new value, which version was created, and who performed the change.

This guide uses a Request entity from a time-off/HRMS application. The current Request state remains in the requests table, while JaVers maintains historical snapshots and commit metadata in its own SQL tables.

2. Dependencies & Configuration

```
<!-- pom.xml -->
<dependency>
  <groupId>org.javers</groupId>
  <artifactId>javers-spring-boot-starter-sql</artifactId>
  <version>7.11.8</version>
</dependency>
```

The SQL Spring Boot starter provides JaVers integration with a relational database and Spring Boot auto-configuration. Use a JaVers release compatible with your Spring Boot/Hibernate version and verify the current version in the official JaVers documentation before production deployment.

```
# application.yml
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/WFM_ENVERS
    username: root
    password: your_password
    driver-class-name: com.mysql.cj.jdbc.Driver

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

For production, prefer controlled database migrations rather than automatic schema updates. The application's existing datasource is used by JaVers' SQL repository.

3. Example Entity & Repository Integration

```
@Entity
@Table(name = "requests")
@Getter @Setter @Builder
@NoArgsConstructor @AllArgsConstructor
public class Request {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;

    @Column(name = "employee_id", nullable = false)
    private Long employeeId;

    private LocalDate startDate;
    private LocalDate endDate;

    @Enumerated(EnumType.STRING)
    private RequestStatus status;

    private Double totalHours;
    private Double totalDays;
}

public enum RequestStatus {
    PENDING, APPROVED, REJECTED, CANCELLED
}
```

```
@Repository
@JaversSpringDataAuditable
```

```
public interface RequestRepository
    extends JpaRepository<Request, String> {
}
```

@JaversSpringDataAuditable is the key Spring Data integration. It enables JaVers auditing around repository operations, so application code can continue using requestRepository.save(...) while JaVers records the resulting object state.

4. How the Auditing Flow Works

```
requestRepository.save(request)
|
+--> JPA updates requests
|
+--> JaVers creates commit + snapshot
|
v
MySQL
requests      -> current business state
jv_commit      -> commit metadata
jv_commit_property -> custom commit properties
jv_global_id   -> object identity/type
jv_snapshot    -> versioned object state
```

Example initial save: employee 1001 submits a request for 15–17 September 2026 with status PENDING. If the manager later changes the status to APPROVED and the entity is saved, JaVers compares the states and records the resulting change.

5. JaVers Tables

Table	Role	Typical data
jv_commit	Commit-level information	commit id, author, timestamp
jv_commit_property	Additional commit metadata	business/correlation properties
jv_global_id	Object identity	entity type + instance id
jv_snapshot	Historical object state	version + serialized property state

Exact columns can vary by JaVers version/configuration. The important distinction is that requests stores the current state, while JaVers tables preserve the audit history.

6. Creating & Updating a Request

```
Request request = Request.builder()
    .employeeId(1001L)
    .startDate(LocalDate.of(2026, 9, 15))
    .endDate(LocalDate.of(2026, 9, 17))
    .status(RequestStatus.PENDING)
    .totalHours(24.0)
    .totalDays(3.0)
    .build();

requestRepository.save(request);

// Later:
request.setStatus(RequestStatus.APPROVED);
requestRepository.save(request);
```

The first save creates the object's initial snapshot. The later save creates another snapshot when JaVers detects the changed state. JaVers can represent property-level changes such as status PENDING → APPROVED.

7. Retrieving Audit History

```
@RestController
@RequestMapping("/api/requests")
@RequiredArgsConstructor
public class RequestController {

    private final Javers javers;

    @GetMapping("/{requestId}/audit")
    public List<Change> getAuditHistory(
        @PathVariable String requestId) {

        JqlQuery query = QueryBuilder
            .byInstanceId(requestId, Request.class)
```

```

        .build();

    return javers.findChanges(query);
}
}

```

GET /api/requests/{requestId}/audit retrieves the changes for one Request instance. `QueryBuilder.byInstanceId(...)` limits the query to the requested entity ID, and `findChanges(...)` returns JaVers Change objects.

8. Snapshots & Version-Based Audit Data

```

List<CdoSnapshot> snapshots = javers.findSnapshots(
    QueryBuilder.byInstanceId(requestId, Request.class).build()
);

```

To group audit changes by JaVers object version:

```

public Map<Long, List<Change>> getRequestChangesByVersion(
    String requestId) {

    List<CdoSnapshot> snapshots = javers.findSnapshots(
        QueryBuilder.byInstanceId(requestId, Request.class).build()
    );

    Map<Long, List<Change>> changesByVersion = new LinkedHashMap<>();

    for (CdoSnapshot snapshot : snapshots) {
        List<Change> changes = javers.findChanges(
            QueryBuilder.byInstanceId(requestId, Request.class)
                .withCommitId(snapshot.getCommitId())
                .build()
        );

        changesByVersion.put(snapshot.getVersion(), changes);
    }

    return changesByVersion;
}

```

Here, `snapshot.getVersion()` identifies the JaVers object version and `snapshot.getCommitId()` identifies the commit associated with that snapshot. This is useful when the UI needs a timeline such as Version 1 → Version 2 → Version 3.

9. Example Version History

```

Version 1
  status    = PENDING
  totalDays = 3.0

Version 2
  status    = PENDING -> APPROVED

Version 3
  totalDays = 3.0 -> 2.5

```

For a production API, it is generally better to map JaVers Change objects to an application DTO instead of exposing JaVers classes directly to Angular.

10. Author & Custom Audit Metadata

An audit trail is more valuable when it identifies the user who made the change. With Spring Security, JaVers can use the authenticated user as the commit author.

```

@Bean
public AuthorProvider authorProvider() {
    return new SpringSecurityAuthorProvider();
}

```

Custom commit properties can also carry business context such as `employeeId`, `requestId`, `source system`, or `correlation ID`.

```

@Bean
public CommitPropertiesProvider commitPropertiesProvider() {
    return domainObject -> {
        Map<String, String> properties = new HashMap<>();

        if (domainObject instanceof Request request) {

```

```

        properties.put("employeeId",
            String.valueOf(request.getEmployeeId()));
        properties.put("requestId", request.getId());
    }

    return properties;
};
}

```

11. Recommended Production Design

```

Angular
|
GET /api/requests/{id}/audit
|
RequestAuditController
|
RequestAuditService
|
Javers
|
JaVers SQL tables

```

- Keep JaVers querying in a service layer instead of putting all audit logic directly in the controller.
- Use DTOs for Angular responses so the frontend is not coupled to JaVers Change/CdoSnapshot classes.
- Configure an authenticated author provider where user identity is required.
- Use custom commit properties for correlation IDs or business context when useful.
- Audit only entities that require historical tracking; audit storage grows with the number of snapshots.
- Protect audit data because snapshots can retain previous values, including values that have since been changed in the business table.
- Use controlled database migrations and integration tests for the JaVers schema in production.

12. End-to-End Checklist

- Add javers-spring-boot-starter-sql.
- Configure MySQL datasource.
- Create the Request entity.
- Add @JaversSpringDataAuditable to RequestRepository.
- Create/update Request through the repository.
- Verify JaVers tables and snapshots.
- Use QueryBuilder.byInstanceId(...) + findChanges(...) for audit history.
- Use findSnapshots(...) for object versions.
- Use snapshot.getVersion() and getCommitId() for version-oriented history.
- Configure author/custom commit properties when required.
- Return a stable audit DTO to Angular.

Conclusion

JaVers provides a clean auditing layer for Spring Boot applications without requiring every entity change to be manually copied into custom audit tables. For the Request module, @JaversSpringDataAuditable handles repository-level auditing, JaVers SQL tables retain historical snapshots, and QueryBuilder APIs provide flexible retrieval by object and version. This approach is suitable for approval workflows, HRMS/time-off systems, and other applications where traceability of business changes is important.

1000-1000-100
1000-1000-100
1000-1000-100
1000-1000-100
1000-1000-100

1990-1995	1995-2000	2000-2005	2005-2010	2010-2015
1990-1995	1995-2000	2000-2005	2005-2010	2010-2015
1990-1995	1995-2000	2000-2005	2005-2010	2010-2015
1990-1995	1995-2000	2000-2005	2005-2010	2010-2015
1990-1995	1995-2000	2000-2005	2005-2010	2010-2015

1990-1999	2000-2009
2010-2019	2020-2029
2030-2039	2040-2049
2050-2059	2060-2069
2070-2079	2080-2089
2090-2099	2100-2109

1990-1999	2000-2009	2010-2019	2020-2029	2030-2039
1990-1999	2000-2009	2010-2019	2020-2029	2030-2039
1990-1999	2000-2009	2010-2019	2020-2029	2030-2039
1990-1999	2000-2009	2010-2019	2020-2029	2030-2039
1990-1999	2000-2009	2010-2019	2020-2029	2030-2039

1990-1991	1991-1992	1992-1993
1993-1994	1994-1995	1995-1996
1996-1997	1997-1998	1998-1999
1999-2000	2000-2001	2001-2002

The screenshot shows a SQL IDE interface. At the top, a toolbar contains icons for file operations, search, and other functions. Below the toolbar, a SQL query is entered in a text area:

```

1 select * from WFM.requests;
2
3 select * from WFM.jv_snapshot;
4
5 select * from WFM.jv_global_id;
6
7 select * from WFM.jv_commit;
8
9 select * from WFM.jv_commit_property;

```

Below the query editor, a toolbar shows the "Result Grid" button, a search filter, and options to edit or export/import data. The "Result Grid" is currently selected, displaying the following table:

snapshot_pk	type	version	state	changed_properties	managed_type	global_id_...	commit_fk
1	INITIAL	1	{ "totalDays": 5.0, "endDate": "...	{ "totalDays": "endDate", "totalHours", "employee...	com.example.java_jaVers.entity.Request	1	1
1	INITIAL	1	{ "totalDays": 5.0, "endDate": "...	{ "totalDays": "endDate", "totalHours", "employee...	com.example.java_jaVers.entity.Request	2	2
3	UPDATE	2	{ "totalDays": 5.0, "endDate": "...	{ "status"	com.example.java_jaVers.entity.Request	2	3
3	UPDATE	3	{ "totalDays": 5.0, "endDate": "...	{ "status"	com.example.java_jaVers.entity.Request	2	4
5	UPDATE	4	{ "totalDays": 5.0, "endDate": "...	{ "status"	com.example.java_jaVers.entity.Request	2	5

At the bottom of the table, there are checkboxes for each column, all of which are currently unchecked.

[illegible]